

The Thiele Machine

A Complete Account from First Principles:
Accountable Computation, Structural Knowledge, and No Free Insight

Devon Thiele

April 2026

Abstract

I'm a car salesman. I started building this in January 2025 because I couldn't find a machine model that correctly accounts for what it costs to know something. Fifteen months later, the result is machine-verified in Coq with zero admitted lemmas, synthesized in hardware, and executable in both extracted OCaml and Python. The central theorem: you cannot certify structural knowledge for free. Any trace that starts uncertified and ends with a certified structural claim must have paid at least one unit in a global monotone ledger called μ . This document explains everything from first principles. What a Turing machine is and what it cannot see. What structure means precisely. What insight means and why it cannot be free. The full 46-opcode instruction set, every group and every opcode. What a categorical morphism is, and why the morphism layer makes this machine strictly richer than any classical machine with the same registers and memory. The hardware, which implements all 46 opcodes in silicon and has formal bisimulation proofs for all 46 of them under their stated preconditions, and why 5 of those proofs require runtime preconditions. The physics connections, which are carefully separated into what is proven and what is conditional. And how to break all of it.

Contents

1	Let me be straight with you	4
2	The Turing Machine: what it can do and what it cannot see	4
2.1	What a Turing machine is	4
2.2	The blind spot	5
2.3	Classical complexity ignores structural cost	5
3	What structure is	6
3.1	Partitions	6
3.2	Axioms on modules	6
3.3	Structural gain	6
4	What insight is, and why it cannot be free	7
4.1	The three tiers of observation	7
4.2	Why insight cannot be free	7
5	The μ ledger	8
5.1	What μ is	8
5.2	Why μ is the unique canonical cost measure	8
5.3	The LASSERT cost formula	9

6	No Free Insight: the central theorem	9
6.1	The abstract version	9
6.2	The specific versions	10
6.3	What this means in practice	10
7	The formal machine state	10
8	The instruction set: all 46 opcodes	11
8.1	Group 1: Partition operations (4 opcodes)	11
8.2	Group 2: Logic and certification (3 opcodes)	12
8.3	Group 3: Compute and data transfer (14 opcodes)	13
8.4	Group 4: Memory and control flow (8 opcodes)	14
8.5	Group 5: I/O, heap, and tensor (7 opcodes)	14
8.6	Group 6: Witness and certification (3 opcodes)	15
8.7	Group 7: Categorical morphism operations (7 opcodes)	15
8.8	Why 46 opcodes?	16
9	What a categorical morphism is: the category theory, from scratch	17
9.1	The idea of a category	17
9.2	Why modules as objects	17
9.3	Why this is not just “extra state”	18
10	Categorical separation: the machine is strictly richer	18
11	The knowledge receipt: what makes certification unforgeable	19
12	How the machine is built: three layers	20
12.1	Layer 1: The Coq kernel	20
12.2	Layer 2: The OCaml extracted runner	20
12.3	Layer 3: The Kami RTL hardware	20
13	CHSH: when correlations break classical explanation	21
13.1	The Bell test setup, from scratch	21
13.2	The CHSH inequality: where it comes from	22
13.3	The quantum bound: why $2\sqrt{2}$?	23
13.4	What the Thiele Machine proves	23
13.5	The REVEAL requirement	23
14	Landauer: information erasure costs energy	24
14.1	Where the formula comes from	24
14.2	What the Thiele Machine proves	24
15	Einstein from computation: what we proved and what we did not	25
15.1	From mu costs to a metric	25
15.2	What is proven	25
15.3	What Theorem 16 actually says	26
16	What Coq is, and why I used it	27
16.1	The Inquisitor	27
17	Zero Admitted: what it actually means	27
18	How to break this	28

19 The full claim ledger	29
20 What I don't know	30
21 Conclusion	30

1 Let me be straight with you

I'm a car salesman. I sell cars for a living.

I started building this in January 2025. I had no CS degree, no math degree, no physics degree. I barely knew what programming was. I used large language models as implementation tools: I specify what I want, the invariants it must satisfy, and the conditions under which it should fail, and I direct the AI to build it. Every Coq proof, every Python file, every line of RTL in this project was generated through that process. The ideas are mine. The proofs compile or they don't, and I verified every one of them.

I'm telling you this because it matters for how I think. I don't come from a background where anyone told me "this is how it works, trust us." I don't have professors or colleagues whose authority I can lean on. When I encounter a claim, I either find the proof or I don't believe it. That is not a methodology I adopted — it is how I think. It's the only way I know.

This document explains everything from the ground up. Not because I think you don't know it, but because I'm not confident I understand something until I can build it from scratch. So I explain it like I'm building it, because I am.

If you are an expert and you find this review of basics annoying, skip to Section 7. The technical content starts there. But if you find anything in the basics wrong, I want to know. Nothing here is correct just because I wrote it.

What this project is. The Thiele Machine is a model of computation where structural knowledge is a first-class citizen with a measurable, auditable cost. The central theorem is called No Free Insight: a computation cannot end in a stronger certified claim than it started with unless the strengthening passed through the machine's explicit cost ledger. This is formalized in Coq and machine-checked. The ledger is called μ . It never goes down. If you go from uncertified to certified, μ went up. But I need to state the scope carefully: raw μ is a receipt that certification work was paid for. It is not, by itself, a universal depth score for the meaning of the claim. The stronger semantic reading only applies when the certification comes with an explicit narrowing witness. That distinction is not cosmetic. It is the hinge between an honest cost ledger and an overclaim.

What this project is not. It is not a claim that I solved P vs NP. It is not a theory of everything. It is not a claim that computation is physics or that physics is computation. The physics connections are real but conditional: they depend on named hypotheses that I have clearly labeled, and I am honest about which parts are proven and which are not. See Section 15.

2 The Turing Machine: what it can do and what it cannot see

Before I can explain what the Thiele Machine does differently, I have to explain what the Turing machine is and exactly where it falls short. Not vaguely falls short — precisely.

2.1 What a Turing machine is

Alan Turing described his machine in 1936 to make precise the intuitive notion of "algorithm." The machine is this:

- An infinite tape. Think of it as an infinitely long strip of paper, divided into cells. Each cell holds one symbol from a finite alphabet, say $\{0, 1, \text{blank}\}$.
- A read/write head. It sits on one cell at a time. It can read the symbol in that cell, write a new symbol over it, and move one step left or one step right.
- A finite set of states. Call them $Q = \{q_0, q_1, \dots, q_n\}$. The machine is always in exactly one state. q_0 is the start state. Some states are designated "halting" states.

- A transition table. For each combination of (current state, symbol under the head), the table says: write this symbol, move this direction, go to this new state.

That is the entire machine. Nothing else.

Here is the remarkable thing: this simple machine can compute *anything that can be computed*. Every algorithm that has ever been written, in every programming language that has ever existed, can be translated into a Turing machine. This is the Church-Turing thesis, and it has never been falsified. To falsify it, you would need to construct a physical device that reliably computes a function that no Turing machine can compute — not faster, but *at all*. No one has done it. Every physical computing device we have ever built turns out to be equivalent to a Turing machine in what it can compute. Your laptop, your phone, every server in every data center — all of them are, in the relevant theoretical sense, Turing machines. They can compute exactly the same class of things. The only difference is speed.

2.2 The blind spot

Here is the precise limitation. The transition table sees exactly two things: the current state q and the symbol under the head. That's it.

The machine cannot ask: “Is this tape sorted?” It has to read every cell.

The machine cannot ask: “Does this graph have two disconnected components?” It has to run a traversal.

The machine cannot ask: “Are these two parts of the input independent of each other?” It has to check.

The Turing machine's view of its tape is LOCAL. One cell at a time. It has no primitive way to see global structure. It can COMPUTE global structure, but computing it and seeing it are different things. To compute that a list is sorted, the machine has to execute a sorting-detection algorithm that touches every element. It pays in time and space for every structural fact it learns.

I spent months staring at this before it clicked. The Turing Machine isn't broken. It's blind by design. It's like trying to find your way through a maze by only ever looking at the floor tile you're standing on. You *can* do it. You will eventually reach the exit. But you're paying for every wrong turn, and you're paying for every part of the maze you already explored. Someone with a map doesn't pay that cost.

The Thiele Machine gives the map. But the map is not free.

2.3 Classical complexity ignores structural cost

Standard complexity theory measures two things: time (number of steps) and space (number of cells or registers used). It says nothing about the cost of knowing that the input has structure.

Consider the satisfiability problem (SAT): given a logical formula ϕ over n boolean variables, is there an assignment that makes it true? In the worst case, a blind machine has to check all 2^n assignments. That's what makes SAT hard.

Now suppose ϕ decomposes cleanly: $\phi = \phi_1 \wedge \phi_2$, where ϕ_1 and ϕ_2 share no variables. You can solve ϕ_1 on its own (2^{n_1} work) and ϕ_2 on its own (2^{n_2} work). Total work: $2^{n_1} + 2^{n_2}$ instead of 2^n . That is an exponential improvement.

But that improvement only applies if you *know* the decomposition exists. And the Turing machine model assigns zero cost to knowing it. The structure is either there or it isn't, and you either see it or you spend time finding it, but the act of certifying “I know this decomposition is valid” is not tracked, not paid for, not audited.

The Thiele Machine says: that act has a cost. Here is how much. Here is where it shows up in the trace. Here is how to verify that the cost was actually paid. This is No Free Insight.

3 What structure is

“Structure” is one of those words everyone uses and nobody defines. I’m going to define it precisely because the whole machine rests on this definition.

3.1 Partitions

Start with the simplest case. Take any set S and divide it into labeled buckets that don’t overlap and cover everything. That is a partition.

Formally: a partition of S is a collection of disjoint non-empty subsets $S_1, S_2, \dots, S_k$ such that $S_1 \cup S_2 \cup \dots \cup S_k = S$. Each S_i is called a module or block or cell, depending on the context.

Now here is the key thing: the partition IS information. Knowing that element x belongs to block S_1 and element y belongs to block S_2 tells you something real. It tells you x and y are in different groups. If you didn’t know the partition, you didn’t know that. If you know the partition, you can look it up in constant time.

In a computational context, partitioning the memory or register state means saying: “These addresses belong to module A, those addresses belong to module B, and these two modules are independent.” That is structural knowledge. A Turing machine has no way to represent this directly. It has to recompute independence from scratch every time it needs to know it. The Thiele Machine stores the partition as part of its state, audits the cost of building it, and enforces that building it was paid for.

3.2 Axioms on modules

A partition alone is just a labeling. What makes it structural knowledge is when you attach *constraints* to the modules. For example: “Module A’s memory region satisfies the predicate Φ_A ,” where Φ_A might say “this region is a valid binary heap” or “all values in this region are between 0 and 100” or “this region encodes a satisfying assignment to formula ϕ .”

These constraints are called axioms. They are written in a formal logic. Concretely, the machine encodes formulas as byte sequences in a standardized prefix-notation format for first-order logic — each operator and operand is written before its arguments, and the whole formula is laid out as a flat byte array in the module’s memory region. (The specific standard the machine uses is SMT-LIB 2.0, a public format; what matters is that it is a fixed, determinate byte encoding with a known bit-length for any given formula.) The machine’s Logic Engine, which runs on-chip as a finite state machine, reads the encoded formula bits from memory and checks whether a certificate validates them. If validation succeeds, the axiom is accepted and the module carries it. If validation fails, the machine sets an error flag.

The crucial point: you cannot add an axiom to a module for free. Adding an axiom means the machine paid to certify it. The axiom is evidence, not assertion.

3.3 Structural gain

A trace has “structural gain” if it starts in a state where some structural fact is not certified and ends in a state where it is. The classic example: a trace that starts with an uncertified partition and ends with a certified satisfying-assignment witness has undergone structural gain.

The Thiele Machine’s core question is: where, in the trace, did the structural gain enter? Not “did the gain happen” — that’s obvious from inspecting the start and end states. But which instruction, at which step, was the one that introduced the certified structural knowledge?

The answer is always: an instruction that costs $\mu \geq 1$. That is No Free Insight.

4 What insight is, and why it cannot be free

In this machine, insight has a precise meaning. I will build up to it.

4.1 The three tiers of observation

Not all observations are the same. Here is the taxonomy, proven in Coq (file `WitnessInsightGeneral.v`):

Tier 0: Raw observation (always free). A raw observation records data without committing to any structural interpretation. The clearest example in the Thiele Machine is a CHSH trial (see Section 13): you run an experiment, you record the outcome in a counter, and nothing structural changes. The counter goes up. The cert channel is not touched. This is free. Cost: 0.

Tier 1: Certified structural insight (always costs ≥ 1). The machine has two ways it records that certification happened. One: a formula was checked and certified — a specific register changes from zero to nonzero. Two: the program explicitly fired the CERTIFY opcode, setting a top-level flag from false to true. Any instruction that flips either of those from “off” to “on” must have cost ≥ 1 . This is proven in Coq (`certification_requires_positive_mu`). Zero Admitted. (The register is called `csr_cert_addr` and the flag is `vm_certified` — both defined precisely in Section 7.)

Tier 2: Certified non-local witness insight (always costs ≥ 1). The sharpest case: a machine that is certified AND whose CHSH statistics show a violation of the classical bound ($S > 2$ — more on what CHSH is in Section 13). Getting certified is already a Tier 1 event (cost ≥ 1). Getting the statistics to show a violation means the data can’t be explained by any shared random strategy — proven in `CHSHStatisticalBridge.v`. Together: certified non-local evidence costs at least 1 to acquire, over any trace of any length.

4.2 Why insight cannot be free

Here is the argument. It is a proof, not an intuition.

Suppose a machine starts uncertified and ends certified. Somewhere in that trace, a specific opcode fired that flipped one of the certification registers from off to on. Either CERTIFY ran and set the top-level certified flag to true, or MORPH_ASSERT ran and proved a morphism exists. Look at the exact step where that happened. That step, by the semantics of the machine, has cost ≥ 1 . And because μ never decreases (proven in `MuLedgerConservation.v`), the final μ is at least $\mu_{\text{before that step}} + 1 \geq \mu_{\text{initial}} + 1$.

This is not a philosophical argument. It is a structural property of the step function. The Coq proof checks it mechanically. The proof does not say “intuitively, certification should cost something.” It says: here is the step function definition, here is the cost semantics, here is the monotonicity lemma, here is the induction on the trace. QED. Zero Admitted.

The deeper reason is this: if you could certify structure for free, the certification would be meaningless. A receipt that any machine can produce at zero cost proves nothing. The whole point of the μ ledger is that it is unforgeable. You cannot accumulate certified structural knowledge without a trace of cost.

But cost alone is not yet semantic depth. A machine can spend resources checking a claim that turns out to be shallow. So in the tightened architecture, LASSERT does not count as structural certification merely because a formula parses and one satisfying assignment exists. It must also carry a non-triviality witness: one satisfying assignment and one falsifying assignment. That proves the asserted constraint excludes at least one state and therefore genuinely narrows the feasible set. The receipt proves spend; the witness proves the spend bought an actual narrowing.

5 The μ ledger

μ (μ) is the global structural cost ledger of the Thiele Machine. Let me explain what it is from first principles.

5.1 What μ is

μ is a natural number attached to every machine state. It starts at 0. It never decreases.

Every instruction carries a cost parameter (δ) in its encoding. The machine adds that cost to μ when the instruction executes. Here is how δ is used in practice:

For **pure compute programs** — programs that do arithmetic, load and store values, and jump — every instruction carries $\delta = 0$. The program I use as the concrete comparison case for the Turing Strictness theorem (`blind_program` in `StructuralAdvantage.v`) has this property: all instructions at $\delta = 0$, and the proof that its total μ -cost is zero is trivial. For these programs, μ stays at 0 unless a cert-setter fires.

For **cert-setter instructions** (`CERTIFY`, `LASSERT`, `MORPH_ASSERT`, `EMIT`, `REVEAL`, `LJOIN`, `READ_PORT`): the machine applies a mandatory wrapper regardless of δ . Even with $\delta = 0$, the cost is at least 1. This is the enforcement that makes No Free Insight work. A real program in `StructuralAdvantage.v` uses `EMIT` with the payload “.” and $\delta = 0$; the proof unfolds that payload into eight concrete bits and confirms it costs exactly 9 ($|\text{payload}|_{\text{bits}} + S(0) = 8 + 1 = 9$). Throughout this document, $S(n)$ means $n + 1$ — it is Coq’s Peano successor function. I use it because the formal proofs are written in Coq and I want the formulas here to match the actual proof terms. $S(\delta) = \delta + 1 \geq 1$ always, which is precisely the enforcement that cert-setters can never be free even at $\delta = 0$. `EMIT`, `REVEAL`, and `READ_PORT` go further than the simple $S(\delta)$ floor: their cost is proportional to the content they carry in actual bits, so certifying more information costs more μ than certifying less.

For **everything that is not a cert-setter** — `ADD`, `LOAD`, `STORE`, `JUMP`, `PNEW`, `PSPLIT`, `PMERGE`, `MORPH`, `COMPOSE`, and the rest — the machine charges exactly δ , and δ should be 0. These operations are structurally free. Creating a partition module, building a morphism chain, doing arithmetic: none of that accumulates μ unless you deliberately set $\delta > 0$, which you should not do. The design principle in `MuCostDerivation.v` (`partition_ops_cannot_cost`) makes this explicit: partition operations are reversible and therefore have zero information cost.

μ only grows when a cert-setter fires. Everything else is free.

The proof that μ is always exactly $\mu_{\text{before}} + \text{instruction_cost}$ for each step (where `instruction_cost` is δ for non-cert-setters; $S(\delta)$ for `CERTIFY`, `MORPH_ASSERT`, and `LJOIN`; $|\text{payload}|_{\text{bits}} + S(\delta)$ for `EMIT`; $\text{flen} \times 8 + S(\delta)$ for `REVEAL` and `READ_PORT`; and $\text{flen} \times 8 + S(\delta)$ for `LASSERT`):

Theorem 1 (μ -Conservation, `MuLedgerConservation.v`, zero Admitted). *For any state s and instruction i : $(\text{vm_apply } s \ i).\mu = s.\mu + \text{instruction_cost } i$.*

This is not a monotonicity claim. It is an equality. μ goes up by exactly the cost of the instruction, always, for every instruction, in every case including error cases. There is no backdoor.

That equality should not be misunderstood. It means the ledger is exact about spend. It does not, by itself, mean every paid unit corresponds to deep semantic gain. In this machine, semantic gain is a stricter notion than spend: it requires a narrowing witness in addition to the ledger entry.

5.2 Why μ is the unique canonical cost measure

Here is the stronger claim, which took me a while to prove: μ is not just A valid cost measure. It is THE cost measure. Any other non-decreasing cost functional that starts at zero and is

consistent with the instruction step semantics must be isomorphic to μ .

Theorem 2 (μ -Initiality, `MuInitiality.v`, zero Admitted). *Let M be any function on machine states satisfying: (a) $M(s_{init}) = 0$ for all initial states, (b) $M(s') \geq M(s)$ for all steps $s \rightarrow s'$, and (c) $M(\text{vm_apply } s \ i) = M(s) + \text{instruction_cost } i$ for all reachable states. Then $M = \mu$ on all reachable states.*

In plain language: there is no alternative accounting. If you want to track structural cost on this machine, you end up at μ . The cost structure was forced by the machine design, not chosen arbitrarily.

5.3 The LASSERT cost formula

The most important cost formula in the machine is for LASSERT, the opcode that certifies a logical formula. Let me build it up:

$$\Delta\mu_{\text{LASSERT}} = \text{flen} \times 8 + S(\text{mu_delta})$$

flen is the encoded formula-unit count, read from the formula's own header in memory. Each unit contributes eight concrete bits, and the per-step certification charge is always at least 1.

Plain version: the machine reads how many encoded formula units are actually present, charges eight bits for each unit, and adds at least 1 for the certification step. A longer formula costs more. A two-unit formula costs at least $16 + 1 = 17$. The programmer does not get to smuggle a larger formula through a smaller bit count.

This is forced, not chosen. You cannot certify a 1000-bit formula by paying for a 10-bit one — the cost counts the formula you actually present. The proof that this is the unique minimum cost satisfying that constraint is in `MuCostDerivation.v` (`cost_necessity + cost_uniqueness`), zero Admitted.

I need to be honest about what this buys you, though. The formula pays for presenting and checking a constraint package. It does not, by itself, guarantee the constraint is meaningful. You could present a formula that is always true — a tautology — and the ledger would still charge you. The machine closes this separately via the dual-witness requirement: see the LASSERT opcode description in Section 8.

The obvious attack, and why it doesn't work. The programmer declares *flen* in the instruction encoding. What stops them from writing *flen* = 0 for a huge formula and paying only $S(0) = 1$?

The hardware catches it. When LASSERT runs, the machine reads the formula's actual encoded-unit count from the formula's own header in memory and compares it to the *flen* in the instruction. If they don't match, the machine traps — error flag set, no certification, no μ advance. This is proven in `StateSpaceCounting.v` (`lassert_honest_cost`, zero Admitted): if LASSERT completes without trapping, the declared count equals the in-memory count, and the μ charge uses the real formula bits.

6 No Free Insight: the central theorem

Now I can state the main theorem precisely.

6.1 The abstract version

Theorem 3 (No Free Insight, `AbstractNoFI.v`, zero Admitted). *Let $\mathcal{M}$ be any machine satisfying:*

- *A3: Machines can go from uncertified to certified only via cert-setter instructions.*

- *A4: Cert-setter instructions have cost ≥ 1 .*

Then for any trace t that takes an uncertified state s_0 to a certified state s_n , the trace contains at least one cert-setter instruction, and $\mu(s_n) \geq \mu(s_0) + 1$.

This theorem is *abstract*: it applies to ANY machine satisfying A3 and A4, not just the Thiele Machine specifically. The Thiele Machine satisfies both axioms by construction. So the theorem applies to it. But it also applies to any future machine you might build that follows the same discipline.

6.2 The specific versions

For the Thiele Machine concretely, there are several specific versions. The two certification channels I mentioned in Section 4 are the formula-certification register (`csr_cert_addr`) and the top-level flag (`vm_certified`). Both are defined precisely in Section 7.

Theorem 4 (No Free Certification, single step, `AbstractNoFI.v §8`, zero Admitted). *If the formula-certification register goes from absent to present in a single step, the instruction that caused it cost ≥ 1 .*

Theorem 5 (No Free Certification, trace level, `AbstractNoFI.v §9`, zero Admitted). *If the formula-certification register was absent at the start of a trace and is present at the end, then the trace paid at least 1 in μ .*

Theorem 6 (No Free Certification via CERTIFY, `AbstractNoFI.v §10`, zero Admitted). *If the top-level certified flag goes from false to true in a single step, the instruction that caused it cost ≥ 1 .*

Theorem 7 (Both channels, `AbstractNoFI.v §11`, zero Admitted). *For ANY transition of either certification channel from absent to present — whether the formula register or the certified flag — μ grows by at least 1.*

Theorem 7 is the master NoFI result. It covers both ways to get certified. There is no third way.

6.3 What this means in practice

Run any program on the Thiele Machine. Watch the μ counter. If the program ends with `vm_certified = true` or with `csr_cert_addr` nonzero, then somewhere in the execution, μ went up by at least 1. You can audit the trace and find exactly where. There is no program that can reach a certified state from scratch without paying for it.

This is what I mean by unforgeable. The μ ledger is the receipt. If you are certified, the receipt exists.

7 The formal machine state

Let me now define the Thiele Machine precisely. I will go through every field.

Definition 1 (Thiele Machine State). *A Thiele Machine state s is a record with 12 fields:*

- $s.\text{vm_pc} \in \mathbb{N}$: the program counter. Points to the next instruction to execute.
- $s.\text{vm_regs} : \mathbb{N} \rightarrow \mathbb{N}$: the register file. The machine has 32 registers indexed by natural numbers 0 through 31. Each register holds a natural number in Coq (unbounded), extracted to a native integer in OCaml, and implemented as a 32-bit word in hardware.

- $s.\text{vm_mem} : \mathbb{N} \rightarrow \mathbb{N}$: data memory. 65536 words. Indexed by natural numbers. Locality requirements: *LOAD* and *STORE* require the active module to be initialized before they work.
- $s.\text{vm_mu} \in \mathbb{N}$: the structural cost ledger. Starts at 0. Never decreases.
- $s.\text{vm_graph}$: the partition graph. Contains modules, edges, morphisms, and a next-ID counter. Defined below.
- $s.\text{vm_err} \in \{\text{false}, \text{true}\}$: the error flag. Set when an instruction fails.
- $s.\text{vm_csrs}$: control-status registers. Contains *csr_cert_addr* (the certification address register, 0 means uncertified) and other machine control values.
- $s.\text{vm_certified} \in \{\text{false}, \text{true}\}$: the top-level certification flag. Set by the *CERTIFY* opcode.
- $s.\text{vm_witness}$: the CHSH witness counters. Eight counters tracking the four outcome combinations for two measurement settings each.
- $s.\text{vm_heap} : \mathbb{N} \rightarrow \mathbb{N}$: the heap. Separate from data memory. Accessed via *HEAP_LOAD* and *HEAP_STORE*.
- $s.\text{vm_output}$: the output buffer. Written by *EMIT* and *WRITE_PORT*.
- $s.\text{vm_imem} : \mathbb{N} \rightarrow \text{vm_instruction}$: instruction memory. Separate from data memory. 65536 instruction slots.

Definition 2 (Partition Graph). The partition graph $s.\text{vm_graph}$ is a record:

- pg_modules : a partial function from module IDs to module records. Each module record holds: *mod_region* (a list of address labels), *mod_axioms* (a list of certified logical constraints), *mod_mu_local* (the module's local μ contribution).
- pg_edges : a list of (source, target) pairs of module IDs recording structural connections.
- pg_morphisms : a partial function from morphism IDs to morphism records. Each morphism record holds: source module ID, target module ID, coupling data, and an is-identity flag.
- pg_next_id : the next fresh identifier for both modules and morphisms.

Remark 1. The partition graph has two distinct layers. The module layer (modules, edges) is the classical structural layer: it records how the state is decomposed into named pieces. The morphism layer (morphisms) records typed categorical relationships between those pieces. These layers are independent. Two machine states can agree on every module and every edge while differing in their morphism maps. This independence is what makes the Categorical Separation Theorem possible (Section 10).

8 The instruction set: all 46 opcodes

The Thiele Machine has 46 opcodes. Every one carries an explicit *mu_delta* parameter. Cost is part of the instruction, not an annotation. I will explain each group from first principles.

8.1 Group 1: Partition operations (4 opcodes)

These opcodes modify the partition graph. They are how the machine builds structural knowledge.

PNEW(R, δ) — allocate a new module.

Creates a new module with region R (a list of addresses), assigns it the next available ID from

`pg_next_id`, increments `pg_next_id`, and charges δ to μ . The region is the set of addresses the module “owns.” Before `LOAD` and `STORE` can touch those addresses, the module must be active and the active-module register must point to it.

Why does this exist? Because structural knowledge needs a home. Creating a module is how you tell the machine “this region of computation is a named unit I am tracking.” The cost δ is programmer-declared and can be 0 — the machine does not force you to pay for allocation itself. If building this partition cost something (you had to figure out the right boundary, run a search, derive the structure), you record that here. If it was trivial, $\delta = 0$ is fine.

PSPLIT(m, L, R, δ) — split one module into two.

Replaces module m with two new modules: one with region L and one with region R . The original module m is removed. Charges δ to μ . This is the refinement operation: you are saying “this region of computation I previously treated as one unit, I am now treating as two independent units with this specific boundary.” The cost δ is programmer-declared. If discovering this boundary required real work, charge for it. If it was bookkeeping, $\delta = 0$ is fine.

PMERGE(m_1, m_2, δ) — merge two modules into one.

Replaces modules m_1 and m_2 with a single new module whose region is the union of their regions. Charges δ (programmer-declared, can be 0). This is the coarsening operation: previously independent pieces are now treated as one. The fact that you know they can be safely merged is structural knowledge — if they had incompatible invariants, the merge would fail. Like `PNEW` and `PSPLIT`, the machine doesn’t enforce a minimum cost here. You charge what you think the insight was worth.

PDISCOVER(r_0, r_1, δ) — query partition structure.

Reads partition state and writes results to registers r_0 and r_1 . Does not modify the partition graph. Advances the PC. Charges δ . This is a read-only observation: you are asking the machine what it already knows about its own structure.

8.2 Group 2: Logic and certification (3 opcodes)

These opcodes interact with the Logic Engine, the on-chip finite-state machine that verifies formulas and certificates.

LASSERT(*freg*, *creg*, *flen*, *cost*) — assert and certify a formula.

Reads a formula whose declared encoded-unit count is *flen* from memory starting at the address in register *freg*. The kernel and hardware both validate that this declared count matches the in-memory formula header before the check is allowed to succeed. A mismatch traps and sets the error flag. Reads a certificate block from memory starting at the address in register *creg*. In the tightened architecture, that block contains two witnesses: one satisfying assignment and one falsifying assignment. The on-chip Logic Engine FSM processes the formula and both witnesses in-place without external solver calls. Validation succeeds only if the declared count is honest, the first witness satisfies the formula, and the second witness falsifies it. That means the formula is not a tautology and really excludes at least one state. `LASSERT` itself is a checked validation step; certification channels are still activated by `CERTIFY` or `MORPH_ASSERT`. Charges $flen \times 8 + S(cost)$ to μ .

This is the most important logic opcode in the machine. It is where a constraint package is checked before any later certification step can lean on it. You cannot fake the pricing anymore by declaring a tiny *flen* for a large in-memory formula: if the header and the instruction disagree, the machine traps. You also cannot fake semantic narrowing with a tautology: the dual-witness requirement closes the tautology-inflation hole because there is no falsifying witness to show the formula excluded any state. The cost formula is still a spend ledger. The honest-length check and the non-triviality witness are what tie that spend to the real checked object.

LJOIN(*c1reg*, *c2reg*, δ) — join two certificates.

Combines two memory-resident certificates into a single composite certificate. The addresses of the input certificates are in *c1reg* and *c2reg*. Charges $S(\delta) \geq 1$. This supports constructing complex multi-part proofs from simpler pieces.

MDLACC(*r*, δ) — MDL cost accounting.

Reads a model description length from register *r* and charges it as a μ cost.

Here is why this opcode exists. Suppose you have two models that both fit the same data. One model is a ten-thousand-parameter neural network. The other is a five-line formula. Both fit the training set. Which one are you buying? The minimum-description-length principle says: the total bill is the model description itself plus the cost of encoding the data once you have the model. The short formula costs more to encode data with, but costs almost nothing to describe. The massive network achieves perfect in-sample compression, but its own description is enormous. The correct choice minimizes the sum.

Translated to the Thiele Machine vocabulary: a module structure that you built has a description-length cost. Not because someone told you it does, but because constructing a specific partition of state space took structure-setting instructions, each of which charged real μ cost. MDLACC is the opcode that lets a program explicitly enter that description cost into the ledger after the structure has been built. It performs no graph change. Its sole effect is to advance the program counter and debit μ by the value in register *r*. The connection to MDL is not a citation to a 1978 paper — it is the claim that the machine’s natural μ -accounting correctly tracks what information-theoretic compression theory says a model costs.

8.3 Group 3: Compute and data transfer (14 opcodes)

Standard arithmetic and data movement. These are the operations you find in every ISA. They mostly carry $\delta = 0$ because they do not introduce new structural knowledge — they just compute.

Opcode	What it does	Typical cost
LOAD_IMM (<i>r</i> , <i>v</i> , δ)	Write immediate value <i>v</i> into register <i>r</i>	0
XFER (<i>r</i> _{dst} , <i>r</i> _{src} , δ)	Copy register <i>r</i> _{src} to <i>r</i> _{dst}	0
ADD (<i>r</i> _d , <i>r</i> _a , <i>r</i> _b , δ)	$r_d \leftarrow r_a + r_b$ (modular 64-bit)	0
SUB (<i>r</i> _d , <i>r</i> _a , <i>r</i> _b , δ)	$r_d \leftarrow r_a - r_b$ (modular 64-bit)	0
MUL (<i>r</i> _d , <i>r</i> _a , <i>r</i> _b , δ)	$r_d \leftarrow r_a \times r_b$	0
AND (<i>r</i> _d , <i>r</i> _a , <i>r</i> _b , δ)	$r_d \leftarrow r_a$ AND <i>r</i> _b (bitwise)	0
OR (<i>r</i> _d , <i>r</i> _a , <i>r</i> _b , δ)	$r_d \leftarrow r_a$ OR <i>r</i> _b (bitwise)	0
SHL (<i>r</i> _d , <i>r</i> _a , <i>r</i> _b , δ)	$r_d \leftarrow r_a$ shifted left by <i>r</i> _b bits	0
SHR (<i>r</i> _d , <i>r</i> _a , <i>r</i> _b , δ)	$r_d \leftarrow r_a$ shifted right by <i>r</i> _b bits	0
LUI (<i>r</i> _d , <i>v</i> , δ)	Load upper immediate: $r_d \leftarrow v \ll 16$	0
XOR_LOAD (<i>r</i> _d , <i>r</i> _a , δ)	Load from absolute address <i>r</i> _a into <i>r</i> _d	0
XOR_ADD (<i>r</i> _d , <i>r</i> _s , δ)	$r_d \leftarrow r_d$ XOR <i>r</i> _s (XOR accumulate)	0
XOR_SWAP (<i>r</i> _a , <i>r</i> _b , δ)	Swap <i>r</i> _a and <i>r</i> _b via XOR (reversible)	0 by convention
XOR_RANK (<i>r</i> _d , <i>r</i> _s , δ)	$r_d \leftarrow \text{popcount}(r_s)$ (Hamming weight)	0

Why does the ISA have all these? Because the Thiele Machine is a complete computing machine, not just a structural accounting system. Programs need to compute intermediate values, manage data, and control flow. The compute group provides all of that. The key property: none of these opcodes touch the certification channels. They cannot create certified structural knowledge.

XOR_SWAP is specifically reversible — it uses the XOR trick to swap two registers without a temporary, and it carries $\delta = 0$ by convention because reversible operations do not increase information entropy.

8.4 Group 4: Memory and control flow (8 opcodes)

Memory access and program flow control.

Opcode	What it does	Note
LOAD(r_d, r_a, δ)	Load <code>vm_mem[r_a]</code> into r_d	Requires active module init
STORE(r_d, r_a, δ)	Store r_d into <code>vm_mem[r_a]</code>	Requires active module init
JUMP(addr, δ)	Set PC to addr unconditionally	—
JNEZ(r, addr, δ)	Jump to addr if $r \neq 0$	—
CALL(addr, δ)	Push return address, jump to addr	Conditional hardware proof
RET(δ)	Pop return address, jump there	Conditional hardware proof
HALT(δ)	Stop execution	—
CHECKPOINT(δ)	Emit checkpoint event	—

LOAD and STORE have a locality requirement: the active module register must be initialized with a valid module ID, and the module must have been initialized with `INIT_PT` and `INIT_ACTIVE_MODULE` before load/store operations work. This enforces that memory access is structurally accounted for: you cannot read or write memory without first claiming ownership of a module region.

8.5 Group 5: I/O, heap, and tensor (7 opcodes)

Input/output and non-register storage.

Opcode	What it does	Cost
READ_PORT(r, bits, δ)	Read <i>bits</i> bits from input into register r	$\text{bits} + S(\delta) \geq 1$
WRITE_PORT(r, δ)	Write register r to output	δ
EMIT($m, \text{payload}, \delta$)	Emit payload as a certified trace event	$ \text{payload} _{\text{bits}} + S(\delta) \geq 1$
HEAP_LOAD(r_d, r_a, δ)	Load from heap at address r_a	δ
HEAP_STORE(r_d, r_a, δ)	Store r_d to heap at address r_a	δ
TENSOR_SET(r_m, r_i, r_v, δ)	Set per-module tensor entry	δ
TENSOR_GET(r_d, r_m, r_i, δ)	Read per-module tensor entry	δ

EMIT and READ_PORT are cert-setters, but their cost goes beyond the simple $S(\delta)$ floor. EMIT charges $|\text{payload}|_{\text{bits}} + S(\delta)$: the machine unfolds the emitted payload into concrete bits and charges for those bits directly, then adds at least 1 for the certification step. A one-byte ASCII payload has 8 payload bits, so with $\delta = 0$ it costs 9. A ten-byte ASCII payload has 80 payload bits, so with $\delta = 0$ it costs 81. The programmer cannot emit more bits for the same price as fewer bits. READ_PORT charges $\text{bits} + S(\delta)$: the number of bits being read from the port, plus at least 1. More information in means more μ paid. The floor is still there — zero-cost certification remains impossible — but now the cost tracks the actual bit content.

TENSOR_SET and TENSOR_GET manage per-module mu-tensor entries, tracking cost-tensor relationships between modules for the physics bridge (Section 15). The hardware contains dedicated hardware-resident tensor table registers (4×4 entries per module). Formal bisimulation proofs exist in `GraphReconstructionBridge.v` under the precondition `tensor_indices_ok i j = true`; unconditional proofs require extending the Kami RTL tensor state representation. For these opcodes the OCaml extracted runner provides the verified reference semantics.

8.6 Group 6: Witness and certification (3 opcodes)

These opcodes interact with the CHSH witness counters and the top-level certification flag.

CHSH_TRIAL(a, b, x, y, δ) — record one CHSH trial.

Updates one of the eight witness counters based on measurement settings ($a \in \{0, 1\}, b \in \{0, 1\}$) and outcomes ($x \in \{0, 1\}, y \in \{0, 1\}$). The counter for the combination (a, b , same/different outcome) goes up by 1. Charges δ to μ .

This is a Tier 0 operation: it records data but does not activate any certification channel. It is provably free in the sense that CHSH_TRIAL does not and cannot trigger a cert-insight event (proven in `WitnessInsightGeneral.v`, `chsh_trial_is_tier0`).

CERTIFY(δ) — top-level proof-carrying certification.

Sets `vm_certified = true`, charges $S(\delta) \geq 1$ to μ , advances PC. This is the simplest cert-channel activation: you are saying “I, the program, certify that this execution has reached an attested state.” The flag is set. The cost is charged. There is no way to set `vm_certified = true` without this opcode, and this opcode always charges ≥ 1 .

REVEAL(r, δ) — disclose a value.

Reveals the value in register r as part of a non-local revelation protocol. REVEAL charges $\text{bits} + S(\delta)$: the number of bits being disclosed, plus at least 1. The cost is also recorded in the μ -tensor at the direction index encoded in the instruction, marking *where* in metric-space the revelation cost lands. Revealing 0 bits still costs at least 1. Revealing 32 bits costs at least 33. You cannot disclose more for the same price as less. This is what makes the CHSH revelation requirement concrete: obtaining non-local statistics requires explicit revelation, and that revelation is priced by how much you reveal.

MORPH_ASSERT (see Group 7 — it activates the cert channel and belongs conceptually here too, but is listed in the morphism group because it operates on a morphism ID).

8.7 Group 7: Categorical morphism operations (7 opcodes)

These are the opcodes that make the Thiele Machine something a Turing machine cannot simulate without losing structure. I will explain what a morphism is before listing the opcodes.

A *morphism* is a record saying: “module A is related to module B in this specific, named way.” Not just “ A and B exist.” Not just “ A and B are connected by an edge.” A morphism has a source module ID, a target module ID, coupling data (a list of (source-cell, target-cell) pairs plus a label string), and an is-identity flag.

The coupling data is actual relational content. MORPH creates a morphism with an empty coupling list initially. COMPOSE computes the relational composition of two morphisms’ coupling lists: pair (a, c) is included if and only if there exists b such that (a, b) is in the first coupling and (b, c) is in the second. This is proven correct in `CategoryBridge.v`. MORPH_TENSOR concatenates two coupling lists. The coupling is not a programmer tag — it is data the machine builds and proves properties about. You can read back the number of coupling pairs via MORPH_GET (selector 2).

Why does this matter? Because two programs can have identical classical state (same registers, same memory, same μ) while having completely different morphism maps. A classical machine cannot distinguish them. The Thiele Machine can, via a single morphism probe. This is the Categorical Separation Theorem (Section 10).

Opcode	Effect	Cost
MORPH(<i>id</i> , <i>s</i> , <i>t</i> , <i>c</i> , δ)	Create morphism <i>id</i> : source = <i>s</i> , target = <i>t</i> , coupling = <i>c</i> . Fails if <i>s</i> or <i>t</i> is not an active module ID.	δ
COMPOSE(<i>id</i> , <i>m</i> ₁ , <i>m</i> ₂ , δ)	Compose morphism <i>m</i> ₁ : <i>A</i> → <i>B</i> with morphism <i>m</i> ₂ : <i>B</i> → <i>C</i> into new morphism <i>id</i> : <i>A</i> → <i>C</i> . Fails if the target of <i>m</i> ₁ ≠ source of <i>m</i> ₂ .	δ
MORPH_ID(<i>id</i> , <i>m</i> , δ)	Create identity morphism on module <i>m</i> : source = <i>m</i> , target = <i>m</i> , is_identity = true.	δ
MORPH_DELETE(<i>id</i> , δ)	Delete morphism <i>id</i> from the morphism map. Sets err if <i>id</i> does not exist.	δ
MORPH_ASSERT(<i>id</i> , δ)	Assert that morphism <i>id</i> exists. If it does: sets <code>csr_cert_addr</code> to nonzero (activates Tier 1 cert channel). If it does not: sets err. EITHER WAY: charges $S(\delta) \geq 1$.	$S(\delta) \geq 1$
MORPH_TENSOR(<i>id</i> , <i>m</i> ₁ , <i>m</i> ₂ , δ)	Form the tensor product of morphisms <i>m</i> ₁ and <i>m</i> ₂ into new morphism <i>id</i> .	δ
MORPH_GET(<i>r</i> , <i>id</i> , <i>field</i> , δ)	Read field (<code>source</code> / <code>target</code> / <code>coupling</code> / <code>is_identity</code>) of morphism <i>id</i> into register <i>r</i> .	δ

MORPH_ASSERT deserves special attention. It charges $S(\delta) \geq 1$ regardless of whether the assertion succeeds. This is the sharpest instance of No Free Insight: the *attempt to claim structural knowledge costs something*, even when the claim is false. A failed assertion is not free. A machine without a structural ledger has no way to express this constraint. In the Thiele Machine it is enforced by the step semantics and proven in **AbstractNoFI.v**.

8.8 Why 46 opcodes?

The number 46 comes from what the machine needs to do:

- Compute arithmetic and data: 14 ops (LOAD_IMM, XFER, ADD, SUB, MUL, AND, OR, SHL, SHR, LUI, XOR_LOAD, XOR_ADD, XOR_SWAP, XOR_RANK)
- Memory and control flow: 8 ops (LOAD, STORE, JUMP, JNEZ, CALL, RET, HALT, CHECKPOINT)
- I/O, heap, and tensor: 7 ops (READ_PORT, WRITE_PORT, EMIT, HEAP_LOAD, HEAP_STORE, TENSOR_SET, TENSOR_GET)
- Partition structure: 4 ops (PNEW, PSPLIT, PMERGE, PDISCOVER)
- Logic and certification: 3 ops (LASSERT, LJOIN, MDLACC)
- Witness and certification flags: 3 ops (CHSH_TRIAL, CERTIFY, REVEAL)
- Categorical morphisms: 7 ops (MORPH, COMPOSE, MORPH_ID, MORPH_DELETE, MORPH_ASSERT, MORPH_TENSOR, MORPH_GET)

$$14 + 8 + 7 + 4 + 3 + 3 + 7 = 46.$$

Every opcode exists for a reason. None are redundant. If you removed CERTIFY, you could not activate `vm_certified`. If you removed MORPH, you could not build morphism relationships. If you removed CHSH_TRIAL, you could not track non-local correlations. The ISA is designed, not grown.

9 What a categorical morphism is: the category theory, from scratch

I said earlier that a morphism is a record saying “module A is related to module B in a specific way.” That is informal. Let me be precise. But I am going to explain category theory from zero, because I did not know what it was when I started this project.

9.1 The idea of a category

A category is a collection of two things: objects and arrows between them.

The objects can be anything. In abstract math, they might be sets, or vector spaces, or topological spaces. In the Thiele Machine, the objects are modules (partitions of state).

The arrows are also called morphisms. An arrow goes from one object to another: $f : A \rightarrow B$ means “there is an arrow f from object A to object B .” The arrow does not have to be a function. In the Thiele Machine, a morphism just says “there is a relationship of this type between these two modules.”

Two laws must hold:

1. **Composition:** If $f : A \rightarrow B$ and $g : B \rightarrow C$, there must exist a composite $g \circ f : A \rightarrow C$. This is what `COMPOSE` implements.
2. **Identity:** For every object A , there must exist an identity arrow $\text{id}_A : A \rightarrow A$. This is what `MORPH_ID` implements.

And composition must be associative: $(h \circ g) \circ f = h \circ (g \circ f)$.

Why composition? Because you want to chain relationships. If A relates to B and B relates to C , you want to say A therefore relates to C , transitively, without having to name every possible A -to- C path individually. Without composition, the structure collapses into a flat list of unconnected pairs. It’s entirely useless for multi-hop reasoning.

Why identity? Because every object needs a minimal self-relationship. An isolated module — no connections to anything else — still needs to be expressible in the framework. `MORPH_ID` is how you say “module A relates to itself with no coupling content.” Without it, isolated objects fall outside the framework entirely, and any module with zero external relationships becomes unrepresentable.

Why associativity? Because the parenthesization of a chain should not matter. Whether you write $(h \circ g) \circ f$ or $h \circ (g \circ f)$, both mean the same three-hop path $A \rightarrow B \rightarrow C \rightarrow D$. If associativity failed, two programs that build the same chain in different groupings would produce different morphisms. That is a bug: the chain is the thing, not how you assembled it.

These laws are proven in Coq in `CategoryLaws.v`. Zero Admitted. Concretely: morphisms carry coupling data — a list of (source-cell, target-cell) pairs plus a label. `COMPOSE` produces a new morphism whose coupling pairs are the relational composition of the two inputs. This is proven in `CategoryBridge.v` (`graph_compose_morphisms_coupling`). The Thiele Machine’s categorical layer is not just called a category — it satisfies the actual axioms on the actual coupling data.

9.2 Why modules as objects

In the Thiele Machine, the objects of the category are the modules of the partition graph. A module is a named piece of the computation: it owns a region of memory or state, it may carry certified axioms about that region, and it contributes a local μ cost.

A morphism between two modules says: “there is a named relationship between module A and module B , with explicit coupling data.” The coupling data is a list of (source-cell, target-cell)

pairs. COMPOSE computes the relational composition of two couplings — pair (a, c) is in the composed coupling exactly when there exists b such that (a, b) is in the first coupling and (b, c) is in the second. This is proven correct. The machine tracks it. Via MORPH_GET (selector 2), you can read back the number of coupling pairs in any morphism.

What the coupling pairs represent is up to the programmer. They could encode:

- Module B refines module A (which cells in A map to which cells in B)
- Module A entangles with module B (which cell-pairs are correlated)
- Module A simulates module B (which states correspond)

The machine enforces the relational composition correctly. The interpretation is yours.

9.3 Why this is not just “extra state”

Here is the question I asked myself for a long time: if you can just add extra registers to a classical machine to store morphism IDs, why is the categorical layer special?

The answer is in the MORPH_ASSERT opcode and in the Categorical Separation Theorem.

Extra registers do not have provenance. You can write anything into a register. You can write 42 into register 5 and claim it is a morphism ID. But the Thiele Machine checks: did a MORPH instruction actually run that created morphism 42 with the source and target you claim? If not, MORPH_ASSERT fails. And MORPH_ASSERT costs $S(\delta) \geq 1$ even if it fails.

Building the morphism chain — calling MORPH, COMPOSE, MORPH_ID — is free. Those ops don’t accumulate μ . What costs is MORPH_ASSERT: the act of asserting that the morphism exists and activating the cert channel. That always costs $S(\delta) \geq 1$. You can build the chain for free, but you cannot certify it for free.

This is why the morphism layer is not “extra state.” It is *provenance-bearing state*. The machine knows where it came from. A classical machine cannot fake this at the same cost. That is the Categorical Separation Theorem.

10 Categorical separation: the machine is strictly richer

Theorem 8 (Categorical Separation, PartitionSeparation.v, zero Admitted). *There exist states s_1 and s_2 such that:*

- $s_1.\text{vm_regs} = s_2.\text{vm_regs}$ (identical registers)
- $s_1.\text{vm_mem} = s_2.\text{vm_mem}$ (identical memory)
- $s_1.\text{vm_mu} = s_2.\text{vm_mu}$ (identical μ)
- $s_1.\text{vm_err} = s_2.\text{vm_err}$ (identical error flag)
- $s_1.\text{vm_pc} = s_2.\text{vm_pc}$ (identical program counter)

yet there exists a single instruction i (specifically MORPH_DELETE with morphism ID k) such that:

- *In s_1 : the instruction succeeds (vm_err remains false)*
- *In s_2 : the instruction fails (vm_err becomes true)*

Construction. Build s_1 : allocate modules A, B, C via PNEW. Create morphism $f : A \rightarrow B$ via MORPH (ID = k). Now s_1 has morphism k in its morphism map.

Build s_2 : load the same values for $\text{regs}[r_0] = A$ and $\text{regs}[r_1] = C$ via LOAD_IMM, padding the μ accumulation to match s_1 ’s μ with cheap structural ops. Module IDs A and C exist. But morphism k was never created.

Both states have identical classical fingerprint: same registers, same memory, same μ , same error flag.

Now apply MORPH_DELETE(k): in s_1 , morphism k exists, deletion succeeds. In s_2 , morphism k does not exist, deletion fails and sets err.

The states were identical by classical observation but are distinguishable by a single morphism probe. $\square$

Corollary 9 (Classical observer is blind, ShadowProjection.v, zero Admitted). *No function of the classical shadow — the tuple ($vm_regs, vm_mem, vm_mu, vm_pc, vm_err, vm_certified$) — can separate all Thiele states. The classical shadow is strictly lossy.*

Corollary 10 (Thiele strictly extends classical, TuringStrictness.v, zero Admitted). *Every classical program leaves pg_next_id unchanged. There exist Thiele programs that change pg_next_id (via PNEW). Therefore Thiele strictly extends classical computation: not just in the sense that it can compute the same things, but in the sense that it can represent and probe states that classical programs cannot reach.*

The classical machine simulates inside the Thiele Machine faithfully (every classical program runs correctly, no side effects on the structural layer). But the Thiele Machine has programs that escape the classical fragment. This is proven, not claimed. Zero Admitted.

11 The knowledge receipt: what makes certification unforgeable

Let me show why the μ ledger is unforgeable by walking through a concrete four-act demonstration.

Act 1: The attempt costs, even when it fails.

Run MORPH_ASSERT(k) where morphism k does not exist. What happens:

- vm_err becomes true (the assertion failed).
- μ increases by $S(\delta) \geq 1$ where δ is the instruction's encoded cost parameter (the attempt cost something).

The machine paid for a failed assertion. A machine without a structural ledger has no way to express this. In the Thiele Machine, the cost of claiming knowledge is charged whether or not the claim is true.

Act 2: Build the chain.

Run this sequence:

PNEW(A) ; PNEW(B) ; PNEW(C) ; MORPH(f, A, B) ; MORPH(g, B, C) ; COMPOSE(h, f, g)

Then MORPH_GET($r_0, h, source$) and MORPH_GET($r_1, h, target$). The resulting state has $r_0 = A$, $r_1 = C$, and morphism $h : A \rightarrow C$ exists in the morphism map. $\mu = 0$ — all these ops are free. The morphism exists, but no structural cost has been paid yet.

Act 3: Certify.

Run MORPH_ASSERT(h). Morphism h exists. The assertion succeeds. csr_cert_addr goes nonzero. Tier 1 cert channel activated. μ goes up by $S(\delta) \geq 1$.

The machine now holds a μ -ledger entry saying: at this point in the trace, a valid composed morphism $A \rightarrow C$ existed, and the machine paid to confirm it. That entry is the knowledge receipt. It is in the ledger. It cannot be removed. It cannot be backdated. μ never goes down.

Act 4: Classical programs cannot forge it.

Program B takes a shortcut: it loads $r_0 = A$ and $r_1 = C$ via LOAD_IMM. Classical state is identical: same registers, same μ (both zero). No morphism h was ever created.

Apply `MORPH_DELETE(h)`: Program A (Act 2) succeeds. Program B fails and sets err.

The μ ledger is not a counter that measures how much work was done. It is a record of which structural commitments were made. Program B did the same amount of “work” (in the μ sense) but did different work. The machine knows the difference.

12 How the machine is built: three layers

The Thiele Machine exists in three forms. They are intended to implement the same semantics. The three layers are tied together by refinement proofs and automated tests.

12.1 Layer 1: The Coq kernel

This is the mathematical ground truth. The Coq kernel defines:

- The `VMState` record (all 12 fields, as in Section 7)
- The `vm_instruction` type (all 46 opcodes as Coq inductive constructors)
- The `vm_apply` function: takes a state and instruction, returns a state. Total. Always returns something.
- All the theorems: NoFI, μ -monotonicity, μ -initiality, categorical separation, Turing strictness, etc.

The Coq kernel operates on natural numbers with no bound. μ can grow to any size. The register file maps any natural number to any natural number. Memory is unbounded. This is the mathematical ideal: infinite resources, like a Turing machine’s infinite tape.

Zero Admitted means: no step in any theorem proof was asserted without derivation. The machine checked everything. I did not.

12.2 Layer 2: The OCaml extracted runner

Coq can automatically extract a Coq development to OCaml. The extracted code is provably equivalent to the Coq specification in the sense that type-correct Coq expressions extract to type-correct OCaml expressions. The OCaml runner executes all 46 opcodes. It handles the morphism graph operations that the hardware does not implement in silicon.

The trust boundary here is named explicitly (not hidden): `ocaml_runner_agrees` is a Section Hypothesis in `OCamlExtractionBridge.v`. This means: the formal claim that the OCaml runner agrees with the Coq semantics is a named hypothesis, not a proven theorem. OCaml semantics are not formalized in Coq — that is the normal trust boundary for any extracted code. CompCert, the verified C compiler, uses the same pattern. The NoFI and μ -monotonicity properties transfer through extraction without needing the hypothesis (they follow from type preservation). The hypothesis is there to make the boundary explicit and auditable.

12.3 Layer 3: The Kami RTL hardware

The Kami framework lets you write hardware specifications in Coq and extract them to Bluespec SystemVerilog. Bluespec SystemVerilog is a high-level hardware description language: you describe what the hardware should compute — the registers, the rules, the conditions under which state transitions fire — and the Bluespec compiler generates standard Verilog RTL. Verilog RTL is the low-level gate-and-register description that synthesis tools can turn into actual logic gates on an FPGA or ASIC. The Thiele Machine hardware is defined in `ThieleCPUCore.v` (Kami module) and extracted through this pipeline.

The hardware has:

- 32 registers, each 32 bits wide (data registers)
- 128-bit instruction encoding (`InstrSz = 128`)
- 65536 words of data memory
- 65536 instruction memory slots
- 64 partition module slots
- 64 morphism descriptor slots
- An on-chip LASSERT FSM for formula verification
- A μ -ALU running in parallel with main execution

All 46 opcodes are implemented in silicon — the RTL in `ThieleCPUCore.v` has circuits for all of them. The question is not which opcodes are in hardware, but which opcodes have a completed formal *bisimulation proof* showing the hardware circuit and the Coq spec agree step-for-step.

Formal bisimulation proofs exist for all 46 opcodes (zero Admitted, all Qed). Specifically:

- 30 opcodes have unconditional proofs (`SupportedOpcode`, `EmbedStep.v`): XFER, LOAD_IMM, LOAD, STORE, ADD, SUB, JUMP, JNEZ, XOR_LOAD, XOR_ADD, XOR_SWAP, XOR_RANK, AND, OR, SHL, SHR, MUL, LUI, HALT, CHECKPOINT, WRITE_PORT, READ_PORT, HEAP_LOAD, HEAP_STORE, CERTIFY, MDLACC, LJOIN, EMIT, PDISCOVER, REVEAL.
- 11 opcodes have conditional proofs (all Qed): PNEW, PSPLIT, PMERGE, LASSERT (under graph-consistency preconditions, `EmbedStep.v` §8), plus MORPH, MORPH_ID, MORPH_DELETE, MORPH_ASSERT, MORPH_GET, COMPOSE, MORPH_TENSOR (under `extended_hw_invariant + abs_full_snapshot`, `GraphReconstructionBridge.v`).

Five opcodes have proofs that require runtime-checkable preconditions and are documented in `RTLGapRegistry.v` (`rtl_gap_count = 5`, proven in Coq):

- 2 driver-managed preconditions: TENSOR_SET, TENSOR_GET (proofs in `GraphReconstructionBridge.v` under `tensor_indices_ok i j = true`; formal unconditional closure requires extending the RTL tensor state representation).
- 3 snapshot preconditions: CALL, RET (proofs under `WellFormedSnapshot`), CHSH_TRIAL (proof under `chsh_bits_ok = true`).

The 46/46 proof coverage (all Qed, zero Admitted) is the correct relationship between an abstract ISA and its physical instantiation. The abstract model defines what is correct; the hardware implements all of it; the formal bisimulation proofs have been completed for all 46 opcodes; five of those proofs carry runtime preconditions that are documented precisely in `RTLGapRegistry.v`. This is not a deficiency. It is honest proof accounting.

The abstract Thiele Machine is the Coq kernel: infinite, complete, proven. The hardware is a finite physical instantiation that covers the most common case. This is the only relationship physical hardware can have with an abstract mathematical model. There is no other kind.

13 CHSH: when correlations break classical explanation

The Thiele Machine has opcodes for recording CHSH trials. I need to explain what CHSH is, and I want to build it from the ground up because the inequality itself is not obvious until you see where it comes from.

13.1 The Bell test setup, from scratch

Imagine two players, Alice and Bob, in separate rooms. They cannot communicate once the game starts. A referee sends Alice one of two questions ($a \in \{0, 1\}$). The referee sends Bob one of two questions ($b \in \{0, 1\}$). Alice answers $x \in \{0, 1\}$. Bob answers $y \in \{0, 1\}$.

The game rule: they want $x \oplus y = a \wedge b$. In plain language: their answers should be different if and only if both questions were 1. For all other question combinations, they should match.

Why 75% is the classical ceiling. Before the game, Alice and Bob can agree on any deterministic strategy: a function $a(x, \lambda)$ for Alice and $b(y, \lambda)$ for Bob, where λ is any shared randomness they agree on in advance. Once the game starts, no communication.

Fix any deterministic strategy. There are four question combinations: (0, 0), (0, 1), (1, 0), (1, 1). The rule says “answer different” only for (1, 1); for the other three, “answer same.” Check every possible output pairing:

Alice output	Bob output	Case (1, 1) wins?
0	0	No ($x = y = 0$, but need $x \neq y$)
0	1	Yes for (1, 1), but now (0, 0), (0, 1), (1, 0) fail
1	0	Yes for (1, 1), but now (0, 0), (0, 1), (1, 0) fail
1	1	No ($x = y = 1$, but need $x \neq y$)

Whatever Alice fixes her output to and whatever Bob fixes his output to, their answers agree for some question combinations and disagree for others. You cannot make them agree for all three “agree” cases and also disagree for the one “disagree” case simultaneously — the outputs are constants, not question-dependent. So the best deterministic strategy wins exactly 3 out of 4 question combinations.

Randomized strategies cannot do better: they are just mixtures of deterministic strategies, and averaging strategies that each win 75% at most gives you 75% at most. This is not a vague claim. It is a direct consequence of linearity of expectation.

So: maximum classical win rate = $3/4 = 75\%$. Proven by exhaustion in `ClassicalBound.v` (zero Admitted): an executable trace using only PNEW, PSPLIT, and CHSH_TRIAL with $\mu = 0$ achieves exactly this, and `MinorConstraints.v` proves no $\mu = 0$ program can exceed it.

13.2 The CHSH inequality: where it comes from

Now here is what took me a while to understand: why is there a specific algebraic combination S that detects whether correlations can be explained classically?

The key move is this. For any fixed shared randomness λ , Alice’s output $A_a(\lambda) \in \{-1, +1\}$ (I’m using ± 1 encoding now, not 0/1) and Bob’s $B_b(\lambda) \in \{-1, +1\}$. Consider the expression:

$$A_0(\lambda)B_0(\lambda) + A_0(\lambda)B_1(\lambda) + A_1(\lambda)B_0(\lambda) - A_1(\lambda)B_1(\lambda)$$

Factor it:

$$= A_0(\lambda)(B_0(\lambda) + B_1(\lambda)) + A_1(\lambda)(B_0(\lambda) - B_1(\lambda))$$

Since B_0 and B_1 are each ± 1 , either $B_0 + B_1 = \pm 2$ and $B_0 - B_1 = 0$, or $B_0 + B_1 = 0$ and $B_0 - B_1 = \pm 2$. The two terms cannot both be large. In either case, $|B_0 + B_1| + |B_0 - B_1| = 2$, and since $|A_0|, |A_1| \leq 1$, the entire expression is at most 2 in absolute value for any fixed λ .

Average over λ :

$$S = \langle A_0 B_0 \rangle + \langle A_0 B_1 \rangle + \langle A_1 B_0 \rangle - \langle A_1 B_1 \rangle$$

where $\langle A_a B_b \rangle$ is the correlation (expected value of the product) when questions are a and b . Since the expression before averaging was bounded by 2 for every λ , the average satisfies $|S| \leq 2$.

That is the argument. The combination with the minus sign is chosen exactly because the factoring trick works: one of the B-terms goes to zero and the other to ± 2 , forcing the whole thing below 2. The inequality $|S| \leq 2$ is not a coincidence. It is a pure algebraic consequence of the outputs being ± 1 and being determined locally.

This is proven in Coq in `ClassicalBound.v` (zero Admitted). The formal statement: for any locally factorizable strategy, $|S| \leq 2$.

13.3 The quantum bound: why $2\sqrt{2}$?

Quantum mechanics allows $|S|$ to reach $2\sqrt{2} \approx 2.828$. Why that number?

In quantum mechanics, Alice's observable for question a is a Hermitian operator $\hat{A}_a$ on a Hilbert space, and similarly for Bob. For a shared quantum state ρ , the correlation is $\langle A_a B_b \rangle = \text{tr}(\rho \cdot \hat{A}_a \otimes \hat{B}_b)$. Now form the CHSH operator:

$$\hat{S} = \hat{A}_0 \otimes \hat{B}_0 + \hat{A}_0 \otimes \hat{B}_1 + \hat{A}_1 \otimes \hat{B}_0 - \hat{A}_1 \otimes \hat{B}_1$$

The maximum of $|\langle \hat{S} \rangle|$ over all quantum states is the operator norm $\|\hat{S}\|$. For qubit observables with eigenvalues ± 1 , this norm reaches exactly $2\sqrt{2}$ when the observables are chosen at the right angles on the Bloch sphere. The Bloch sphere is a way to represent any qubit state geometrically: a qubit $\alpha|0\rangle + \beta|1\rangle$ with $|\alpha|^2 + |\beta|^2 = 1$ corresponds to a point on a unit sphere, with the two classical states $|0\rangle$ and $|1\rangle$ at opposite poles. A Hermitian observable with eigenvalues ± 1 corresponds to an axis through the sphere. The angle between two observables is the angle between their axes. The optimal CHSH setting places Alice's axes 45 degrees apart and Bob's axes 45 degrees from Alice's, giving a total separation in the operator algebra that drives the maximum eigenvalue of $\hat{S}^2$ to 8. You get $2\sqrt{2}$ from the Pythagorean theorem applied to a 2-dimensional operator algebra: the maximum eigenvalue of $\hat{S}^2$ is 8, so $\|\hat{S}\| = \sqrt{8} = 2\sqrt{2}$.

That bound is proven in this repository in `TsirelsonUpperBound.v`: no quantum strategy exceeds $2\sqrt{2}$. And it is tight: the quantum model achieving it is in `TsirelsonQuantumModel.v`. Zero Admitted throughout.

The gap between 2 and $2\sqrt{2}$ is real. No locally factorizable strategy can bridge it. That is the signature of quantum non-locality: correlations that cannot be produced by any shared classical randomness, no matter how much pre-game coordination you allow.

The Inquisitor's note on the rational approximation. Because Coq works in exact arithmetic, the Tsirelson bound in the formal proofs uses the rational approximation $5657/2000 \approx 2.8285$, which is a verified upper bound on $2\sqrt{2}$. This is not an error or a weakness. It is how you work with irrational numbers in a verified formal system.

13.4 What the Thiele Machine proves

Theorem 11 (Classical CHSH Bound, `ClassicalBound.v`, zero Admitted). *For any locally factorizable strategy (Alice's answer depends only on a and λ ; Bob's answer depends only on b and λ ; λ is shared randomness), $|S| \leq 2$.*

Theorem 12 (Non-locality of CHSH violation, `CHSHStatisticalBridge.v`, zero Admitted). *WitnessCount configurations with $S > 2$ cannot be produced by any local hidden variable model.*

Theorem 13 (Tsirelson upper bound, `TsirelsonUpperBound.v`, zero Admitted). *For the symmetric CHSH configuration, $|S| \leq 2\sqrt{2}$.*

What does this have to do with computation? The connection is through the witness counters and the certification layer. If a Thiele Machine program accumulates CHSH trial results (via

CHSH_TRIAL) and then certifies that $S > 2$ (via CERTIFY or LASSERT), the certification is non-free. Certifying non-local statistics requires paying $\mu \geq 1$. The statistics themselves (raw CHSH trials) are free. Claiming you know what they mean is not. This is the witness insight taxonomy (Section 4).

13.5 The REVEAL requirement

Getting $S > 2$ in any computational context requires revelation. Here is why that is not just a policy: it is a structural fact about the step semantics.

Alice’s CHSH_TRIAL increments Alice’s witness counters based on her question a and her outcome x . Bob’s CHSH_TRIAL increments Bob’s witness counters based on b and y . The cross-correlations $\langle A_a B_b \rangle$ cannot be computed without combining both sides’ results. The only opcode that explicitly exports state across that boundary is REVEAL. Without a REVEAL step, the joint correlation cannot be computed from a single side’s counters.

This is proven in `RevelationRequirement.v` (zero Admitted): if execution starts with `csr_cert_addr = 0` and ends with `csr_cert_addr  $\neq$  0`, then some structure-setting step was executed. Trying to get certified non-local statistics above the classical bound without paying for revelation is not just inadvisable. It is structurally impossible.

14 Landauer: information erasure costs energy

Rolf Landauer proved in 1961 that erasing one bit of information from a physical system must release at least $kT \ln 2$ joules of heat into the environment, where k is Boltzmann’s constant and T is the temperature of the environment. I want to explain where that formula comes from because it matters for how the μ ledger connects to physics.

14.1 Where the formula comes from

A bit of information is stored in a physical system with two distinguishable states: call them 0 and 1. The system might be a tiny magnet pointing up or down, a capacitor charged or uncharged, a transistor on or off.

“Erasing” the bit means forcing the system to state 0 regardless of what it was before. Before erasure, the bit could be 0 or 1 — you have no idea which. After erasure, you know it is 0. You have gone from maximum uncertainty (1 bit of entropy) to zero uncertainty.

Here is the heat physics. Total entropy of the universe must not decrease (second law of thermodynamics). The bit’s entropy went down by $k \ln 2$ (that is what “1 bit = $\ln 2$ nats of entropy” means in physical units). Some other part of the universe must gain at least that much entropy to compensate. The cheapest way to dump entropy is to release it as heat into a heat bath at temperature T . Dumping $\Delta S = k \ln 2$ of entropy into a bath at temperature T requires releasing heat $Q = T \cdot \Delta S = kT \ln 2$.

That is Landauer’s argument. It is not complicated. Information is physical. The bit has to go somewhere. Destroying it means the physical system’s entropy decreases, and the second law guarantees you pay for that in heat.

I found this argument important because it is the same kind of argument as No Free Insight: you cannot get structure for free, and the machine tracks where you paid. The difference is that Landauer’s principle is about physical energy and the μ ledger is a formal accounting tool. The connection between the two is real but conditional.

14.2 What the Thiele Machine proves

Theorem 14 (μ -Landauer Step Bound, `LandauerDerivation.v`, zero Admitted). *For any step that erases k bits of information, the instruction’s μ -delta is $\geq k$.*

This is a theorem about the formal step semantics of the machine. It says: the machine’s cost accounting is consistent with Landauer’s constraint. If you erase information, you pay in μ .

The physical interpretation — that μ costs correspond to actual thermodynamic entropy — is conditional. It depends on a named hypothesis (`mu_landauer_unruh_calibrated`) that calibrates μ to physical energy units. That hypothesis is physically motivated and consistent with the formal development, but it is not proven in Coq. It is a bridge to physics, not a derived fact. I say this directly because it is true.

The formal content of `LandauerDerivation.v` is narrower and cleaner: (1) only `CERTIFY` sets `vm_certified := true` — proven by case analysis on every instruction; (2) `CERTIFY`’s cost is $S(\delta) \geq 1$ by construction; (3) therefore certification always requires positive μ -cost; (4) total μ -cost bounds total irreversible bit-operations in the trace. None of this requires a physical calibration. Physical calibration would plug in the Boltzmann constant and the temperature, turning μ counts into joules. That step is the named bridge hypothesis.

15 Einstein from computation: what we proved and what we did not

The Thiele Machine’s μ costs define a geometry. Here is what that means, from first principles.

15.1 From mu costs to a metric

Each module in the partition graph carries a local μ cost. Two modules connected by an edge in the graph are “adjacent” in a discrete structure. The μ costs give that structure a metric: the cost of going from one module to an adjacent one is the μ difference between the two states.

A discrete metric defines curvature. To see why: on a flat surface, if you draw a triangle, the interior angles sum to 180° . On a curved surface — the surface of a sphere, for example — the angles sum to more than 180° . Curvature is the amount by which triangles deviate from flat. The μ metric defines triangles on the partition graph, and those triangles have a measurable angle-sum.

The Euler characteristic χ is a topological invariant of the simplicial complex — a number that depends on the overall shape of the space, not its geometry. For a triangulation with V vertices, E edges, and F faces: $\chi = V - E + F$. For a sphere, $\chi = 2$. For a torus (a donut surface), $\chi = 0$. The Gauss-Bonnet theorem says the total curvature of a closed surface equals $2\pi\chi$: the geometry and the topology are linked.

15.2 What is proven

Theorem 15 (Discrete Gauss-Bonnet, `DiscreteGaussBonnet.v`, zero Admitted). *For a finite simplicial complex induced by the partition graph’s μ -metric, the sum of discrete Gaussian curvatures equals $2\pi\chi$, where χ is the Euler characteristic of the complex.*

This is a fully proven theorem about the discrete geometry defined by μ costs. Zero Admitted. The chain from this to Einstein’s field equations goes through five steps:

1. Gauss-Bonnet (proven: `DiscreteGaussBonnet.v`)
2. Clausius entropy-area relation (proven: `ClausiusFromEntropyArea.v`)
3. Raychaudhuri equation (proven conditionally: `DiscreteRaychaudhuri.v`)
4. Jacobson derivation of Einstein equations from thermodynamics (chain assembled in `EinsteinEmergence.v`)
5. Full tensor Einstein equations $G_{\mu\nu} = \kappa T_{\mu\nu}$ (proven in section: `EinsteinEquationsFull.v`)

Let me explain steps 3 and 4 from first principles, because they are the non-obvious ones.

The Raychaudhuri equation. Imagine two nearby light rays traveling through space. They start parallel. As they travel, they can converge (focus toward each other) or diverge (spread apart). The rate of focusing is called the “expansion” of the beam.

The Raychaudhuri equation says: the rate of change of the expansion depends on the Ricci curvature of the spacetime along the beam direction. Positive Ricci curvature means the beam converges. This is gravitational focusing — matter curves spacetime, spacetime curves geodesics, geodesics converge. This is how gravity works in General Relativity.

In the Thiele Machine, the discrete analog is: the null congruence expansion (how fast adjacent null geodesics of the μ -metric converge) decreases when the effective Ricci curvature is positive. `DiscreteRaychaudhuri.v` proves this for the μ -geometry. The result is conditional on a named hypothesis (`lorentzian_coupling_positive`) because the discrete geometry is Euclidean by construction and requires an explicit sign choice to be treated as Lorentzian. That sign choice is physically motivated but not proved from the Coq definitions alone.

The Jacobson derivation. Ted Jacobson showed in 1995 that if you apply the first law of thermodynamics ($dE = T dS$) to the boundary of every small causal region of spacetime, and if entropy is proportional to the *area* of that boundary, then Einstein’s field equations follow as a thermodynamic equation of state. Why area and not volume? Jacob Bekenstein argued in 1973 that the information in a region of space is bounded by its boundary area, not its volume — the interior degrees of freedom cannot exceed what can be encoded on the surface at one bit per Planck-area patch. Stephen Hawking confirmed this for black holes: a black hole’s entropy is exactly proportional to its event horizon area ($S = A/4G\hbar$ in natural units). This is non-obvious and surprising. Intuitively volume should hold more information than surface. The answer is that gravity compresses the interior: if you tried to pack more entropy into a region than its boundary allows, the region would collapse into a black hole. The surface is the ceiling. In the Thiele Machine’s formal development, this is a named hypothesis (Bekenstein saturation): the claim that the partition graph’s boundary area bounds its interior information content is taken as a bridge premise, not derived.

The key move: gravity is not fundamental. It is what you get when you treat spacetime as a thermodynamic system. The Einstein equations say that the geometry of spacetime adjusts so that entropy is extremized — just as pressure equilibrates in a gas. You do not need quantum mechanics or strings. You need thermodynamics applied to the geometry.

The Thiele Machine’s μ costs play the role of entropy. The partition graph’s morphism layer plays the role of the causal boundary. `JacobsonBridgeComponents.v` makes the two components explicit: a Clausius-type relation (first law for the μ -geometry horizon) and a Raychaudhuri-to-Einstein link (focusing implies curvature constraints). Together, they imply the Einstein target. `EinsteinEmergence.v` assembles the chain.

Theorem 16 (Full Tensor Einstein Equations, `EinsteinEquationsFull.v` §10, zero Admitted in section). *For all $\mu, \nu \in \{0, 1, 2, 3\}$:*

$$G_{\mu\nu}(s, sc, v) = \kappa \cdot T_{\mu\nu}(s, sc, v)$$

15.3 What Theorem 16 actually says

Theorem 16 is conditional. It is proven inside a Coq `Section` with three named hypotheses:

- `diagonal_metric_h`: the μ -metric is diagonal (no off-diagonal metric terms).
- `diagonal_efe_h`: the diagonal Einstein equations hold (already proven separately).
- `off_diagonal_ricci_zero`: the off-diagonal Ricci curvature vanishes.

These are not **Admitted** proofs. They are explicitly named hypotheses that make the trust boundary auditable. The proof of Theorem 16 from these three hypotheses is zero Admitted: the derivation is valid. The hypotheses themselves are physically motivated (the off-diagonal Ricci vanishes in vacuum for symmetric metrics, and in the continuum limit of many discrete geometries) but not proven in Coq.

The honest claim: the Thiele Machine’s μ geometry is consistent with Einstein’s field equations in the diagonal metric case, and the extension to the full tensor case is proven conditional on the off-diagonal Ricci hypothesis. This is not a claim that gravity *is* computation. It is a claim that the formal analog is consistent.

If you want to know when the hypotheses fail: any situation where the μ -metric has strong off-diagonal correlations between modules would stress-test `off_diagonal_ricci_zero`. Designing an experiment to distinguish the μ -geometry from standard GR is an open problem. I have not solved it.

16 What Coq is, and why I used it

Coq is a proof assistant. It is a programming language where the type system is so expressive that you can write proofs as programs and have the compiler check whether they are valid.

Here is the idea. In a normal programming language, a type says something about what kind of value a variable holds. In Python, `x: int` means x is an integer. In Coq, types are propositions. The type `n > 0` means “the proof that n is positive.” A function from `n > 0` to `n + 1 > 0` is a proof that if n is positive, $n + 1$ is too.

The compiler checks whether the types line up. If your proof has a hole — if you write “this step is obvious, trust me” — the compiler rejects it. There is no way to convince the compiler of a false theorem by being persuasive or authoritative. The only way through is to actually prove it.

I used Coq because I do not trust informal arguments. Including my own. I spent months being convinced by my own reasoning that various claims were true, only to find when I tried to write the Coq proof that some key step was wrong. The machine caught it. That is the point.

16.1 The Inquisitor

Beyond Coq’s type checker, the project has a custom proof auditor called the Inquisitor (`scripts/inquisitor.py`). It runs on every commit and checks:

- No **Admitted** anywhere in the active proof files
- No circular proofs (theorems that depend on themselves through the import graph)
- No vacuous theorems (theorems that prove things like `True` or `0 = 0` but are claimed to be content)
- No tautological implications (theorems of the form $P \rightarrow P$)
- No smuggled constraints (definitions that assume what they should be proving)

Current Inquisitor status: zero HIGH/MEDIUM/LOW findings across all 188 Coq files.

17 Zero Admitted: what it actually means

“Zero Admitted” means: every theorem in the proof tree was derived without any gaps. No step was asserted without a derivation. The machine verified the complete chain.

This is what it does NOT mean:

- It does not mean the theorems prove physical reality. The theorems prove things about the Thiele Machine model, which is a formal object in Coq. Whether the model correctly describes physical reality is a separate empirical question.
- It does not mean the Coq axioms themselves are consistent. Coq is based on the Calculus of Constructions — a formal language where types and programs are the same thing, so a proof is literally a program and a proposition is literally a type. Within this system, constructing a term of a given type means constructing a proof of the corresponding proposition. This foundation is believed to be consistent but this has not been proven within Coq itself (Gödel’s incompleteness theorem prevents any sufficiently expressive system from proving its own consistency).
- It does not mean the hardware implements exactly the Coq semantics in all cases. All 46 opcodes have Qed bisimulation proofs; five carry runtime preconditions documented in `RTLGapRegistry.v`.
- It does not mean the OCaml runner is bitwise identical to the Coq semantics. The trust boundary is the named hypothesis `ocaml_runner_agrees`.

The kernel tier (Section 6) is the strongest tier. The theorems listed there have zero Admitted and zero named hypotheses. They are machine-checked derivations from the Coq type theory axioms and the Thiele Machine definitions. Nothing else.

The bridge tier involves named hypotheses (`mu_landauer_unruh_calibrated`, `off_diagonal_ricci_zero`, `ocaml_runner_agrees`). These are explicitly labeled and auditable. They are not Admitted proofs. They are documented trust boundaries, which is how honest formal development works.

18 How to break this

Every claim in this document has a falsification condition. Here are the main ones.

Falsify No Free Insight.

Find a Thiele Machine program trace that:

- starts with `vm_certified = false` and `vm_mu = 0`
- ends with `vm_certified = true`
- ends with `vm_mu = 0`

If you find such a trace, the Coq proof of `certification_requires_positive_mu` is wrong and the compilation will fail when you add the counterexample as a theorem.

Falsify μ -monotonicity.

Find a state s and instruction i such that $(\text{vm_apply } s \ i).\mu < s.\mu$. The Coq proof of `vm_apply_mu` will fail.

Falsify categorical separation.

Prove that for ALL states s_1, s_2 with identical classical shadow, ALL instructions produce the same error behavior. This would contradict `categorical_separation` in Coq, meaning the proof has an error.

Falsify Turing strictness.

Prove that every Thiele program can be simulated by a classical program with the same observable behavior (registers, memory, μ) without ever touching the partition graph or morphism map. This would contradict `D5_thiele_strictly_extends_classical`.

Falsify the classical CHSH bound.

Exhibit a locally factorizable strategy that achieves $|S| > 2$. This would contradict `chsh_stat_violation_not_1` and also violate known mathematics.

Stress-test the physics bridge.

Design an experiment where a physical system implementing the Thiele Machine's μ -geometry produces off-diagonal Ricci curvature. If `off_diagonal_ricci_zero` fails empirically, the conditional Einstein theorem `full_tensor_efe_conditional` loses its grounding. The theorem itself would still be valid (it is still true that the hypothesis implies the conclusion), but the hypothesis would be known false.

The Inquisitor standard.

Run `python scripts/inquisitor.py` on the repository. If it finds any HIGH/MEDIUM/LOW finding, the proof hygiene has degraded. Current status: zero findings. If that changes, it is a bug.

19 The full claim ledger

These are the kernel-tier claims (zero Admitted, zero named hypotheses):

1. **no_free_certification**: `csr_cert_addr` going $0 \rightarrow \text{nonzero}$ in one step requires cost ≥ 1 .
2. **no_free_certification_mu**: same, with μ increment ≥ 1 .
3. **no_free_certification_trace_mu**: trace-level version.
4. **no_free_certification_certified**: `vm_certified` going `false` $\rightarrow$ `true` requires cost ≥ 1 .
5. **certification_requires_positive_mu**: both channels covered.
6. **abstract_nfi**: the abstract version for any machine satisfying A3+A4.
7. **universal_nfi**: `abstract_nfi` plus cost lower bound.
8. **thiele_nfi_pc_indexed**: PC-indexed execution version.
9. **mu_is_initial_monotone**: μ is the unique canonical cost measure.
10. **vm_apply_mu**: exact ledger identity $\mu' = \mu + \delta$.
11. **kernel_certified_implies_positive_mu**: from zero, reaching certified implies $\mu > 0$.
12. **categorical_separation**: classical-identical states distinguishable by morphism probe.
13. **classical_observer_cannot_separate**: no classical function separates morphism-distinct states.
14. **shadow_proj / shadow_separation_theorem**: formal surjection, strictly lossy.
15. **D2_faithfulness**: classical programs embedded faithfully in Thiele.
16. **D3_conservativity**: classical programs leave structural layer untouched.
17. **D4_strictness**: there exist Thiele steps that escape the classical fragment.
18. **D5_thiele_strictly_extends_classical**: Thiele strictly exceeds classical semantics.
19. **degenerate_projection_theorem**: shadow projection is the classical equivalence quotient.
20. **no_classical_certification_decider**: no classical machine can decide morphism-certification.
21. **chsh_stat_violation_not_local**: CHSH-violating statistics are non-local.

22. **partition_refinement_nonfree**: certified partition knowledge cannot be free.
23. **witness_insight_nonfree_general**: certified non-local witness insight requires $\mu \geq 1$.
24. **witness_insight_complete_taxonomy**: full three-tier taxonomy proven.

Bridge-tier claims (zero Admitted, with named hypotheses or documented trust boundaries):

- **Hardware bisimulation**: 46/46 opcodes with Qed proofs (30 unconditional, 11 conditional, 5 with runtime preconditions); documented in `RTLGapRegistry.v`.
- **OCaml extraction**: `ocaml_bisimulation_closure` with named `ocaml_runner_agrees` hypothesis.
- **Cost formula forcing**: `cost_necessity + cost_uniqueness`, conditional on Landauer-Unruh calibration for physical interpretation.
- **Einstein equations**: `full_tensor_efc_conditional` with named `off_diagonal_ricci_zero` hypothesis.

20 What I don't know

This document is honest about what is proven. Here is what I don't know.

Physical interpretation of μ . The formal development is consistent with Landauer's principle and with the Einstein equation structure. Whether the μ ledger actually corresponds to thermodynamic entropy in a physical Thiele Machine is an open empirical question. I think it does. I cannot prove it.

Complexity implications. The No Free Insight theorem says structural certification costs $\mu \geq 1$. It does not resolve P vs NP. It does not give a new complexity separation. It gives a machine model where structural cost is tracked, which is a different and more fine-grained kind of accounting than time complexity. What implications this has for the complexity landscape is an open question.

Quantum mechanics connection. The CHSH and Tsirelson results are proven for the machine model. Whether a physical Thiele Machine can actually produce CHSH-violating statistics (exploiting quantum entanglement) depends on hardware design and quantum engineering that is outside the scope of this project.

Off-diagonal Ricci. The condition `off_diagonal_ricci_zero` is physically reasonable (it holds in vacuum for symmetric metrics) but I have not proven it in Coq for the general μ -geometry case. This is the main open gap in the physics bridge.

I could pretend these are answered. I could wave at them vaguely and say "future work." I don't know. These are real open questions.

21 Conclusion

That's the whole thing.

A Turing machine is blind to global structure. It can compute anything but it pays in time whenever it needs to discover structure that is already there. The Thiele Machine makes structural knowledge visible, first-class, and cost-bearing. The one rule is No Free Insight: you cannot certify stronger claims than you started with without paying in a monotone ledger μ that records every structural commitment.

The machine has 46 opcodes. Most of them are ordinary compute operations that carry zero structural cost. The structural cost is concentrated in a small set of opcodes: `LASSERT` (formula certification), `CERTIFY` (global certification flag), `MORPH_ASSERT` (categorical existence claim), and the partition operations (`PNEW`, `PSPLIT`, `PMERGE`). These are the

opcodes that can transition a machine from uncertified to certified, and none of them can do it for free.

The categorical morphism layer is strictly richer than any classical equivalent. Two programs with identical registers, memory, μ , and error flag can be distinguished by a single morphism probe. No classical observer can see this difference. The Coq proof checks it.

The hardware implements all 46 opcodes in silicon. Formal bisimulation proofs — Coq proofs that the hardware step function matches the Coq spec step-for-step — have been completed for all 46 of them (zero Admitted). Five proofs carry runtime preconditions: COMPOSE and MORPH_TENSOR under `extended_hw_invariant`; TENSOR_SET and TENSOR_GET under `tensor_indices_ok`; CALL, RET, CHSH_TRIAL under `WellFormedSnapshot` or `chsh_bits_ok`. These are documented in `RTLGapRegistry.v` (`rtl_gap_count = 5`).

The physics connections — Landauer, Einstein, CHSH, Tsirelson — are real but carefully scoped. Some are proven. Some are conditional. None are claimed to be physics. They are formal analogs that are consistent with physics, and I have been honest about which is which.

Zero Admitted. Zero Inquisitor findings. Every claim has a falsification condition. If this is wrong, it will fail visibly, and that is exactly how it should be.

I'm a car salesman. I built this. Read the proofs.